



ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE
DEPARTMENT OF COMPUTER AND COMMUNICATION SCIENCES

COM-599
MASTER PROJECT IN COMMUNICATION SYSTEMS

Graph Convolutional Networks for Skills Management

Furkan Karakaş

With GASPAR Number: 306399

Academic Supervisor
Asst. Prof. Dr. Tanja Käser
Machine Learning for Education Laboratory
tanja.kaeser@epfl.ch

In-Company Supervisor
Dr. Igor Kulev
F. Hoffmann-La Roche AG
igor.kulev@roche.com

ML4ED



Master Project Autumn, 2021-2022

March 11th, 2022

Contents

1	Introduction	3
2	Theoretical Background	3
2.1	Graph Convolutional Networks	4
2.2	Text Graph Convolutional Networks	4
3	Datasets	6
4	Results	8
4.1	Document Classification	9
4.1.1	Roche dataset	11
4.1.2	Dicecom dataset	11
4.2	Embeddings	13
4.2.1	Roche dataset	15
4.2.2	Dicecom dataset	15
5	Improvements and Modifications	23
6	Conclusion and Future Work	24
	References	25

Abstract

Large companies in today's world have a huge employee basis. The employees usually register the skills that they possess via an online portal. This registration process can take place when they apply for the open position for the first time, and when they start working in the company, they can register extra skills that they gained throughout their career. It is of utmost importance for a company to be able to make sense of the available skills that an employee has, which skills are connected to each other, and which employees are similar to each other in terms of skills that they possess. This information can be utilized in a clever way by the people in human resources department in a company in order to facilitate the recruitment process of new employees. For example, if a skill is found to be missing, which is valuable to a company, then the proper employment decisions can be taken based on this information.

This is a collaborative project between Roche and Swisscom. From Roche's side, a graph-based approach is utilized in order to tackle this problem whereas from Swisscom's side, a language-based approach is used. A novel concept called "graph convolutional networks" (GCN) is used to model the employees and their skills in an undirected graph. In this report, the theory behind the GCN, the network used to train the model, the used datasets and the obtained results are explained. Different approaches and modifications to the original network are discussed, which improve the performance of the results. Later, the shortcomings of the current GCN implementation are discussed, and some further points are suggested as next steps in the future work.

1 Introduction

Data science has been playing an important role in today’s world more than ever before. Making sense of the available data becomes crucial for the success of the organizations, in particular, for large companies. They usually have many employees working in different departments in the company, who are coming from different backgrounds and have acquired various different skills throughout their career. The success of a company depends on the ability that the management team is able to keep track of the available skills in the company, which skills are missing, and which employment decisions have to be made in order to narrow the gap between available skills and the desired presence of skills in a future project or in a work group. This is exactly the aim of this project: meaningfully represent the skills of employees in a company so that similar skills can be grouped together. This way, the workforce planning can be optimized in the company. Whenever a task concerning a particular skill is performed, one can investigate similar skills to that skill and if one or many of those similar skills are missing, then the HR department can appoint next employee to the task or a corresponding recruitment decision can be taken by the company, where the next candidate would ideally possess the missing skills.

One important fact to consider is that the skills registered in the system can be present in many different forms. Some skills could have low frequency, or some particular skills could be very frequent among the employees. That is the reason why we need to do data preprocessing and clean the data first before using a machine learning model. This whole pipeline will be explained in section 3.

This is a collaborative project between Roche and Swisscom. We want to tackle the same problem by using different approaches. We considered two approaches to this problem: language-based and graph-based. From Roche’s side, we wanted to investigate the graph-based approach. The motivation to utilize graphs to tackle this problem is that you can naturally represent the employees and the skills that they possess as nodes in a graph, and you can draw edges between employees and the skills that they possess. Furthermore, the underlying data are simply comma-separated skills that the employees possess, i.e., the ordering of the skills does not matter and there is no syntactic nature of the underlying text. We use a concept called *graph convolutional networks* (GCN), which has recently gained a lot of popularity. [Kipf and Welling, 2017] presented an approach for semi-supervised learning on graph-structured data, which can learn hidden-layer representations of the nodes. The motivation for their research is that datasets can come in an unstructured way like in an arbitrarily-structured graph. Being able to process such datasets is an important concept and has applications in many fields such as chemistry with molecule classification and text classification in documents.

In section 2, we explain the theoretical background with the GCN.

2 Theoretical Background

Utilizing industry skills in a proper way has attracted attention lately due to the novelties in the natural language processing and deep learning areas. Word embedding techniques are used to represent the words with a vector of fixed size. Word2vec [Mikolov et al., 2013] is a popular semi-supervised machine learning algorithm to create a vector embedding from a large volume of corpus of text. Another research paper inspired by this idea is skill2vec [Van-Duyet et al., 2019], which uses a similar technique as in word2vec, applying it to the employee skills field in order to find the correct candidate for a job position, who possesses the appropriate set of skills¹.

In this project, we will follow a similar approach by representing a word as a vector of fixed size by means of a machine learning method called *graph convolutional networks*, which operates on an arbitrary dataset

¹The Github repository <https://github.com/duyet/skill2vec-dataset> contains datasets of various sizes and trained model for skill2vec.

structured as a graph.

2.1 Graph Convolutional Networks

A GCN [Kipf and Welling, 2017] is a convolutional, multi-layer deep learning network, which operates directly on arbitrarily-shaped graphs. Formally, a graph is a tuple $G = (V, E)$, where V is the set of vertices (or nodes) in the graph, and the size of the set is $|V| = n$. E is the set of edges between two vertices. If two vertices v_1 and v_2 are connected to each other, then the edge $(v_1, v_2) \in E$. In this implementation, we are only considering undirected graphs, hence $(v_2, v_1) \in E$ and those edges have the same weights. We assume that every node is connected to itself, i.e. $(v, v) \in E$ for all $v \in V$. These self loops are an important assumption in the structure of the network, which we will explain it later. From the graph, we can naturally introduce *the adjacency matrix* $A \in \mathbb{R}^{n \times n}$, where $A_{ij} \in \mathbb{R}$ is the weight corresponding to the edge $(v_i, v_j) \in E$. Note that due to the undirected graph, we have $A_{ij} = A_{ji}$ for all $1 \leq i, j \leq n$, and, due to the self loops, we have $A_{ii} = 1$ for all $1 \leq i \leq n$. From this adjacency matrix, we introduce *the diagonal degree matrix* $D \in \mathbb{R}^{n \times n}$, which is defined as:

$$D_{ij} = \begin{cases} \sum_{k=1}^n A_{ik}, & \text{if } i = j \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

In a GCN, a node in the graph will be able to capture the information present in its immediate neighbors. When multiple layers of GCN are stacked, the network will be able to capture the information in the multiple order neighborhoods.

The output from the first layer of the GCN $L^{(1)} \in \mathbb{R}^{n \times k}$ is calculated as:

$$L^{(1)} = \rho(\tilde{A}XW_0) \quad (2)$$

where $W_0 \in \mathbb{R}^{n \times k}$ is the weights to be trained in the network and k is the number of hidden units in the first layer. $X \in \mathbb{R}^{n \times n}$ is the input feature matrix of the nodes. If there is a specific relationship between the nodes, you can simply represent that relationship in this matrix. Otherwise, each node is simply related to itself and there will be no predefined relationship between the nodes, i.e., input feature matrix is the identity matrix, $X = I$. $\tilde{A} \in \mathbb{R}^{n \times n}$ is the normalized symmetric adjacency matrix which is calculated as $\tilde{A} = D^{-\frac{1}{2}}AD^{-\frac{1}{2}}$. [Kipf and Welling, 2017] stated that symmetric normalization is more interesting for the network, that is the reason why the degree matrix is multiplied from both sides with square roots instead of just multiplying from a single side. ρ is an activation function, e.g. $\text{ReLU}(x) = \max(x, 0)$. We will be using ReLU and softmax activation functions in this project.

Equation 2 represents the first layer in a GCN. This can be extended to multiple layers as:

$$L^{(j+1)} = \rho_j(\tilde{A}L^{(j)}W_j) \quad (3)$$

where j represents the current layer, and $L^{(0)} = X$.

As we mentioned before, inclusion of self loops in the adjacency matrix is important. When we demonstrate the neighbors of a node as entries in the adjacency matrix, we allow the information about those neighbors to pass to the next layer in the GCN. However, since we do not have self loops included, the information about the node itself will not be able to pass to the next layer in the network. That is why self loops must be added to the adjacency matrix, which corresponds to adding ones across the diagonal.

2.2 Text Graph Convolutional Networks

One particular application of GCN is classification of textual documents. In this application, the number of nodes $|V| = n$ is the number of documents plus the number of unique words in the corpus.

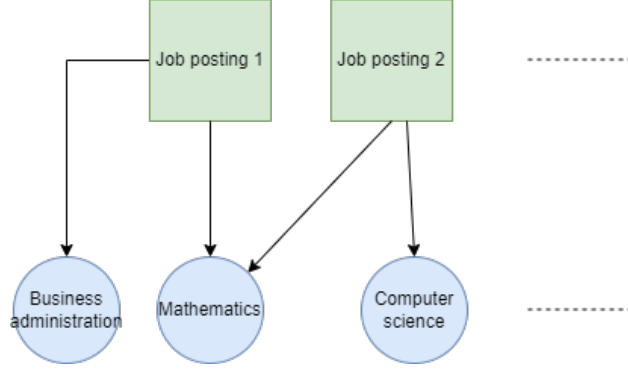


Figure 1: A sample representation of the graph. The squares represent the documents and the circles represent the unique words. Note that in the original implementation, the words are tokenized and they are presented separately unlike in this illustration.

Figure 1 is an example how the graph looks like in a text GCN. The edges between documents and words need to be built accordingly. In order to relate different words to each other, artificial edges between words will also be created. [Yao et al., 2018] suggested different methods to create the entries in the adjacency matrix based on whether the nodes represent documents or words in the graph. They proposed to use sliding windows with a fixed size of 20 across the corpus and calculate the *pointwise mutual information* (PMI) in order to create the artificial edges between words. The entries in the adjacency matrix can be described as follows:

$$A_{ij} = \begin{cases} \text{PMI}(i, j) & \text{if } i, j \text{ are words and } \text{PMI}(i, j) > 0 \\ \text{TF-IDF}_{ij} & \text{if } i \text{ is document, } j \text{ is word} \\ 1 & \text{if } i = j \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

TF-IDF stands for *term frequency-inverse document frequency*. It takes the frequency of the occurrence of a word in the document into account, but it also penalizes that frequency if the same word occurs in multiple documents. The PMI value of two words i, j is calculated as:

$$\text{PMI}(i, j) = \log \frac{p(i, j)}{p(i)p(j)} \quad (5)$$

$$p(i, j) = \frac{\#W(i, j)}{\#W} \quad (6)$$

$$p(i) = \frac{\#W(i)}{\#W} \quad (7)$$

In Equation 7, the numerator represents the number of sliding windows that contain the word i and the denominator represents the total number of sliding windows across the corpus. Similarly, in Equation 6, the numerator represents the number of sliding windows that contain both of the words i, j .

After the graph is built with respect to the formulae in Equation 4, the graph is fed into a two-layer GCN. The second layer has the same number of neurons as the label set in the documents. The final activation function is chosen to be a softmax layer in order to return the probabilities of each individual class. From Equation 3, the overall network looks like this:

$$Z = \text{softmax}(\tilde{A}\text{ReLU}(\tilde{A}XW_0)W_1) \quad (8)$$

$\tilde{A}$ is the same as in Equation 2. The loss function of the network is defined as the famous *cross-entropy* function:

$$\mathcal{L} = - \sum_{d \in \mathcal{D}} \sum_{f=1}^F Y_{df} \ln Z_{df} \quad (9)$$

where $\mathcal{D}$ is the text corpus, F is the size of label set, Y is a binary matrix which indicates if the given document d has the label at f , and Z is the result of the softmax layer from Equation 8. The weight parameters W_0 and W_1 can be trained via gradient descent with some predetermined epochs. As the optimizer in the training step, the Adam optimizer [Kingma and Ba, 2017] with the selected learning rate is used, the other parameters being the default ones.

3 Datasets

To accomplish the task of representing the skills available in the industry, we need to have an actual dataset which consists of employees in various companies in the industry. For the sake of this project, it is not allowed to work with a synthetic dataset where skills of employees will be generated by an a priori probability distribution for the sake of testing. That is the reason why an actual dataset is needed which contains the entries of and information about real people or jobs. However, such a dataset is not easily available in most cases since these data contain highly sensitive information.

The end goal of this project is to represent skills and classify documents in a dataset. We need appropriate datasets which are suitable for these end goal tasks. That is the reason why we need to have clear labels of the documents in the dataset. If there are no predefined labels, then we need to create those in a clever way. Furthermore, since skills are distinct entries, we should consider the skills that have some acceptable frequency in the dataset. For example, investigating skills that appear only once in the entire dataset does not make much sense for the purpose of this project.

In order to experiment with the model, two different datasets will be used. First one is the Workday dataset of the employees in Roche, which contains highly confidential information. The employees register their skills in the website via comma-separated values, and it is visible in the website which job family group they belong to. This job family group is the target variable in this dataset. The second dataset is a publicly available dataset, which contains various U.S. Technology Jobs on Dice.com².

The Dicecom dataset contains the name of the job position such as "senior software engineer", "application engineer" and "security engineer"; the description of the job and an additional column called "skills" to indicate the skills needed for this job position. However, the skills available in this column are not standardized and they are available in many different formats. Some examples of those data are:

1. "Big data" AND Pig AND Spark AND Python AND Hive AND SQL AND Hbase AND mapreduce AND
2. "Business Analysis", "Fixed Income", Derivatives. "Asset Management", LPC, Intex, "1010 Data"
3. "Business Analyst" Ecommerce Agile
4. SEE BELOW

²This dataset is hosted in Kaggle for the purpose of data science competitions and can be accessed under the link <https://www.kaggle.com/PromptCloudHQ/us-technology-jobs-on-dicecom>.

5. Null
6. (See Job Description)
7. .net; c#; asp.net; mvc; RESTful; WebAPI's; SQL; mongodb

Table 1: Some statistics of the datasets available in this project. Note that Dicecom dataset does not have a structured, comma-separated skills available in the dataset. Hence, the skills statistics are recorded with NA.

Dataset	#Entries	Missing	Total Skills	Unique Skills
Dicecom	22,000	223	NA	NA
Roche Workday	103,549	61,840	761,608	62,446

Some statistics of these two datasets are shown in Table 1. In the Dicecom dataset, the rows have been recorded as missing if the skills column contains "null", "see below" or "see job description".

In the Dicecom dataset, there is "jobtitle" column, which indicates the name of the job corresponding to the entry in the dataset. However, these jobs are not standardized. To get distinct labels out of this dataset, we run a string matching algorithm to get the 6 most frequent jobs from the dataset. When we count the values of the job titles, we get the following top 6 entries in the dataset:

1. Java Developer: 171
2. Project Manager: 144
3. Network Engineer: 127
4. Software Engineer: 118
5. Business Analyst: 115
6. .Net Developer: 70

Then we run a string matching regex on the job titles in order to get 6 distinct labels in the dataset, which are "java", "project", "network", "software", "business" and ".net". The rest of the data will be removed from the dataset. The final count of the labels is shown in the list below with a total of 7148 entries:

1. software: 1642
2. java: 1432
3. .net: 1240
4. business: 1060
5. project: 996
6. network: 778

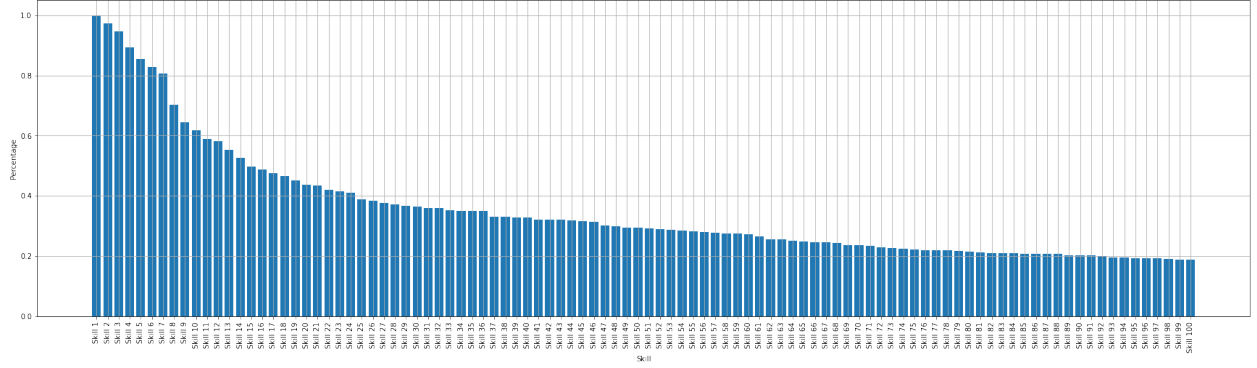


Figure 2: The distribution of top 100 skills in the Roche dataset. The skills are sorted by descending order.

The Roche employee dataset is also processed. Job titles which have fewer than 10 occurrences are removed from the dataset. Also, some job titles had mostly irrelevant skills and they made the training of the graph neural network worse by providing very low evaluation scores in the test dataset. Those entries were also removed from the dataset.

Figure 2 shows the percentage of top 100 skills in the Roche dataset. For confidentiality reasons, names and numbers of the exact skills are not given. In the set of unique skills, some skills appeared only once. Since they would not have any meaning for the GCN, the skills which appear fewer than 50 times are also removed from the Roche dataset.

Figure 3 shows the number of employees, who own the given number of skills. We can observe more or less a power law in this distribution. We also realize that one worker registered around 145 skills in their profile, which does not look like a very realistic scenario.

In the original Roche dataset, there are 23 different job family groups, which will be the labels in the machine learning model. Number of samples in these groups are shown in Figure 4. Again, for confidentiality reasons, the exact name of the job family groups is not given.

To prevent the imbalance of classes during training, job family groups fewer than 100 entries within are removed from the dataset. Furthermore, the job family groups 12, 15, 16 and 18 are also removed since the evaluation values of these classes were relatively lower than other classes. The reason could be that the employees in these departments had very diverse skills, and it was difficult to predict them based on the skills that they had. After the complete data processing, the number of data points is reduced from 103,549 to 13,137 which makes 12.69% of the initial data.

4 Results

To assess the results obtained from the GCN, we use a baseline model of a simple TF-IDF calculation on the documents. The resulting vectors, which represent the documents, are fed into a logistic regression classifier with one-vs-rest scheme [Pedregosa et al., 2011]. We split the documents into train and test sets with a 75-25 ratio. The split is done regarding the class balances in the train and test sets. Furthermore, 10% of the train set will be used as a validation set. At the end of each epoch during training, the model is validated with this validation set. If the result of the validation accuracy does not improve over the last 50 epochs, which is the early stopping number, the training stops and the model is then evaluated on the final test set.

In order to tune some hyperparameters of the model, the Python library called `hyperopt` is used

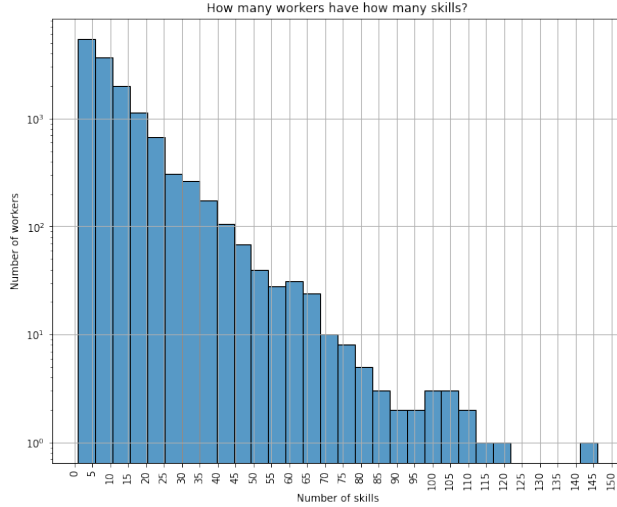


Figure 3: The distribution of number of skills that an employee has in the Roche dataset. The x-axis is in linear scale, and the y-axis is logarithmic scale with basis 10.

[Bergstra et al., 2015]. The hyperparameters to tune the model were learning rate of the network, number of hidden neurons in the hidden layer, dropout and weight decay of the parameters. It was observed that even increasing the weight decay a tiny bit made the results much worse. Hence, weight decay is simply set to zero and it is not used in the hyperparameter optimization.

Table 2: Search space and the best parameters found after running the hyperparameter optimization with the Python library `hyperopt`. The type "uniform" represents that the hyperparameter can take any value in that interval uniformly whereas the type "choice" represents that the hyperparameter can take only discrete values in that interval.

Hyperparameter	Type	Lower	Upper	Step	Best
Learning rate	Uniform	0.01	0.1	NA	0.012
#Hidden neurons	Choice	100	500	50	150
Dropout	Uniform	0	0.5	NA	0.4646

The result of the hyperparameter optimization is shown in Table 2. Though, it is worth mentioning that the test accuracy results are all close to one another and there was no significant changes when the hyperparameters are modified.

In the next subsections, we show the classification and embedding results.

4.1 Document Classification

In this section, we will talk about the results of document classification for both Roche and Dicom datasets.

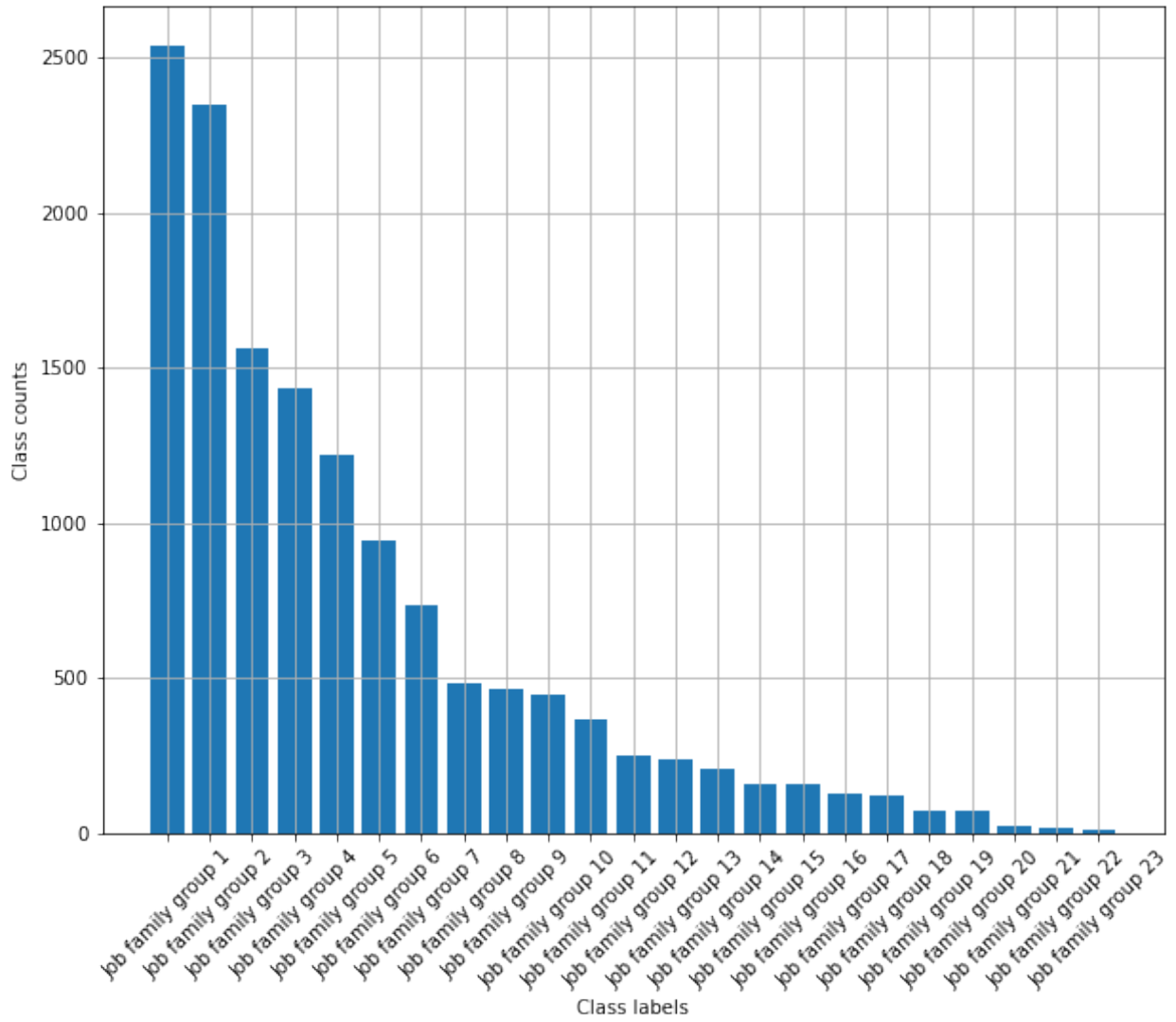


Figure 4: The distribution of the class labels in the Roche dataset.

4.1.1 Roche dataset

Dummy model. It is usually a good idea to include a dummy classifier to see how the results would be if there was no training at all. We include a dummy classifier, which predicts the label of the data point based on the probability distribution of the labels in the training set (stratified). The results of this dummy classifier is shown in Table 3.

Table 3: Classification report of the dummy classifier (stratified) for the Roche dataset.

	Precision	Recall	F1-score	Support
Job family group 10	0.02	0.02	0.02	121
Job family group 3	0.13	0.13	0.13	389
Job family group 4	0.12	0.11	0.11	375
Job family group 9	0.03	0.03	0.03	118
Job family group 5	0.10	0.10	0.10	299
Job family group 6	0.06	0.07	0.06	221
Job family group 11	0.03	0.03	0.03	86
Job family group 7	0.05	0.04	0.05	205
Job family group 14	0.02	0.02	0.02	52
Job family group 8	0.03	0.03	0.03	116
Job family group 13	0.00	0.00	0.00	63
Job family group 1	0.18	0.20	0.19	597
Job family group 2	0.17	0.17	0.17	601
Job family group 17	0.00	0.00	0.00	42
accuracy			0.12	3285
macro avg	0.07	0.07	0.07	3285
weighted avg	0.11	0.12	0.11	3285

Baseline model. The classification report of the baseline TF-IDF model is shown in Table 4. There are in total 14 different labels, and in general, precision score seems to be higher than the recall one. For example, the class "Job family group 14" has 81% precision score from the test set but only a recall value of 25%, meaning that many samples of label "Job family group 14" could not be correctly labeled as so. This phenomenon can more easily be seen in Figure 5 – not many other classes than "Job family group 14" were labeled as "Job family group 14" (high precision) but many of the true "Job family group 14" samples have been labeled as other classes (low recall).

Graph convolutional network. The classification report of the graph convolutional network model is shown in Table 5. As in the baseline model, precision scores seem to be higher than recall ones. In terms of accuracy and macro & weighted average scores, the baseline and GCN models have almost the same performance where GCN has a slightly better performance. After testing the GCN model for 10 rounds, the there was an accuracy value of 68% 7 times and 67% 3 times, which can help us conclude that the GCN model has the upper hand in terms of accuracy by +1%.

4.1.2 Dicecom dataset

Dummy model. As in subsubsection 4.1.1, we include a dummy classifier for the Dicecom dataset, which predicts the label in the test dataset by investigating the class distribution in the train dataset (stratified).

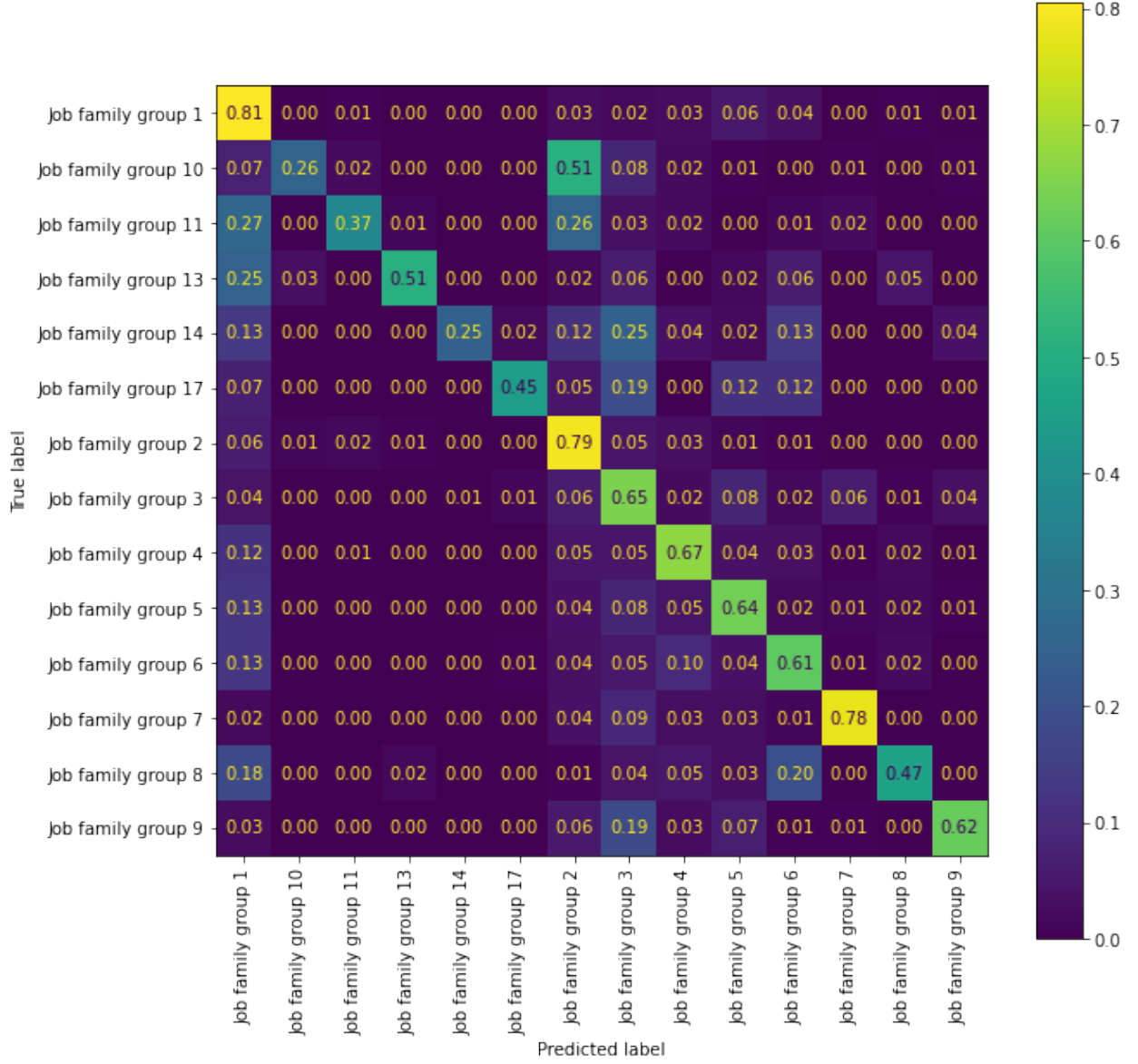


Figure 5: The confusion matrix for the baseline TF-IDF model for the Roche dataset. X-axis represents the predictions, y-axis represents the true labels. The numbers are normalized with respect to the true labels (row).

Table 4: Classification report of the baseline TF-IDF model for the Roche dataset.

	Precision	Recall	F1-score	Support
Job family group 10	0.74	0.26	0.38	121
Job family group 3	0.58	0.65	0.61	389
Job family group 4	0.71	0.67	0.69	375
Job family group 9	0.74	0.62	0.67	118
Job family group 5	0.61	0.64	0.62	299
Job family group 6	0.59	0.61	0.60	221
Job family group 11	0.60	0.37	0.46	86
Job family group 7	0.80	0.78	0.79	205
Job family group 14	0.81	0.25	0.38	52
Job family group 8	0.64	0.47	0.54	116
Job family group 13	0.78	0.51	0.62	63
Job family group 1	0.66	0.81	0.72	597
Job family group 2	0.72	0.79	0.75	601
Job family group 17	0.68	0.45	0.54	42
accuracy			0.67	3285
macro avg	0.69	0.56	0.60	3285
weighted avg	0.67	0.67	0.66	3285

The results of the dummy classifier can be found in Table 6.

Baseline model. The same baseline model as in subsection 4.1.1 is used in order to train the model with the Dicecom dataset. The plot of the confusion matrix for the baseline model is shown in Figure 6. The classification report is shown in Table 7. It is clear that the performance of the baseline model is better on the Dicecom dataset than the Roche one.

Graph convolutional network. As the final part of the document classification, we will investigate the GCN on the Dicecom dataset. The classification report of the graph convolutional network model is shown in Table 8. Unlike in the Roche dataset, the baseline model yields a higher accuracy value than the GCN one in the Dicecom dataset with a margin of 2%.

4.2 Embeddings

In this section, we will talk about the results of document, word or skill embedding that we got from the models for both Roche and Dicecom datasets. One important point to mention is that there are no word embeddings for the baseline TF-IDF model but only document embeddings, because in TF-IDF, words do not have their own separate vector representations.

In order to plot the embeddings, we use a tool called *t-distributed stochastic neighbor embedding* [van der Maaten and Hinton, 2008]. The final number of components is 2, which corresponds to the components in the x and y axes. The initialization of the algorithm is chosen as "random".

As for the GCN model, we investigate only the second-layer embeddings since they provide better results than the first-layer ones.

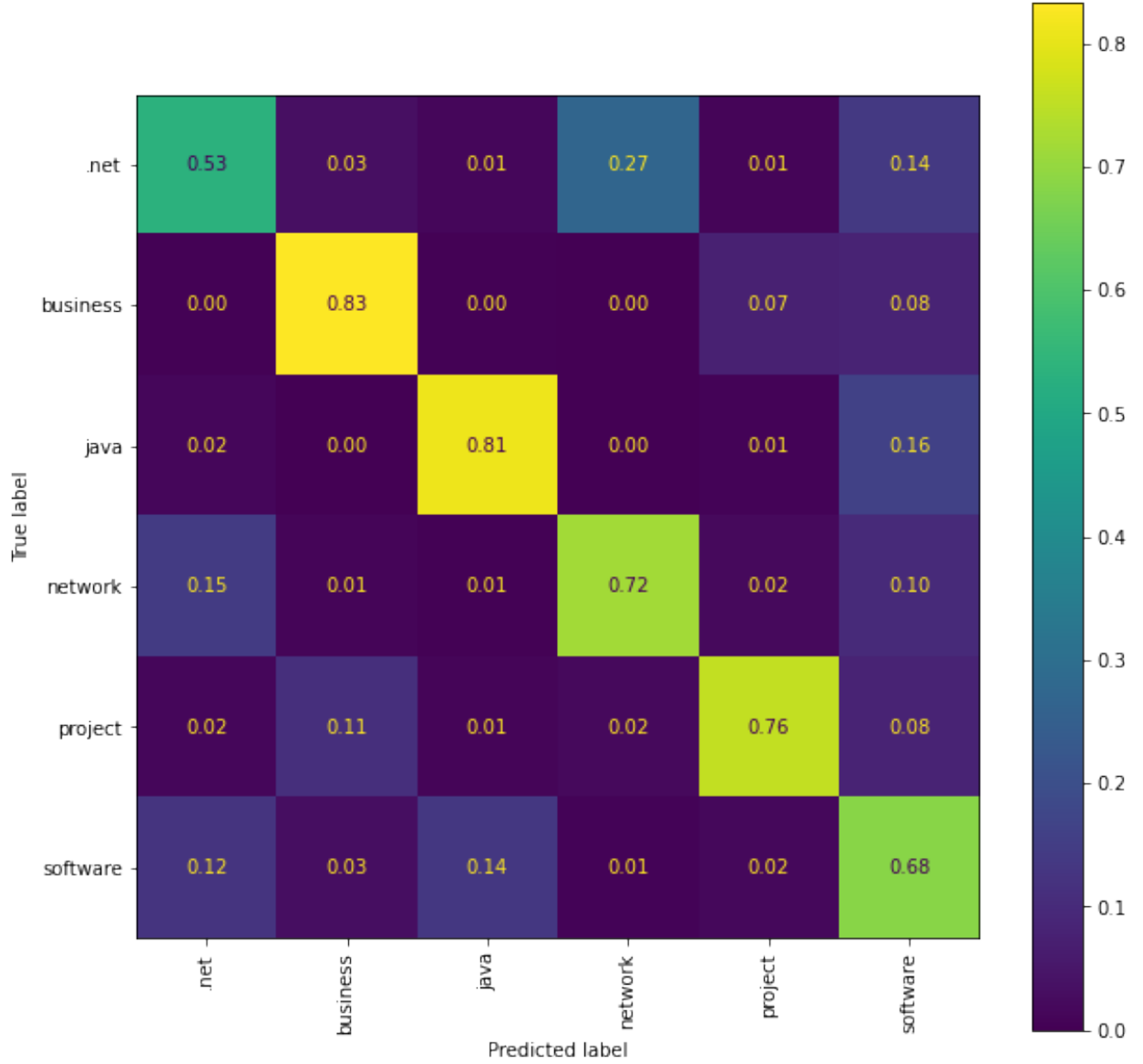


Figure 6: The confusion matrix for the baseline TF-IDF model for the Dicecom dataset. X-axis represents the predictions, y-axis represents the true labels. The numbers are normalized with respect to the true labels (row).

Table 5: Classification report of the GCN model for the Roche dataset.

	Precision	Recall	F1-score	Support
Job family group 10	0.60	0.32	0.42	121
Job family group 3	0.57	0.71	0.63	389
Job family group 4	0.67	0.73	0.70	375
Job family group 9	0.65	0.67	0.66	118
Job family group 5	0.64	0.62	0.63	299
Job family group 6	0.57	0.62	0.59	221
Job family group 11	0.56	0.41	0.47	86
Job family group 7	0.78	0.75	0.77	205
Job family group 14	0.70	0.27	0.39	52
Job family group 8	0.67	0.45	0.54	116
Job family group 13	0.70	0.51	0.59	63
Job family group 1	0.73	0.77	0.75	597
Job family group 2	0.75	0.76	0.76	601
Job family group 17	0.63	0.40	0.49	42
accuracy			0.67	3285
macro avg	0.66	0.57	0.60	3285
weighted avg	0.67	0.67	0.67	3285

4.2.1 Roche dataset

Baseline model. The t-SNE visualization of the document embeddings of the baseline model applied to the Roche dataset is shown in Figure 7. Initial number of dimensions of the document vectors is 726. As we can see in the visualization, the documents are not really separated from each other well and there are no clear clusters.

Graph convolutional network. The initial number of dimensions of the document vectors is 14, which is the same as the number of distinct classes in the dataset.

The skill embeddings of the GCN model applied on the Roche dataset is shown in Figure 8. For confidentiality reasons, the names of the skills will not be mentioned in this report. From the visualization, it is apparent that similar skills are grouped together to form clusters. That way, related skills can be detected and appropriate decisions in the company can be taken so that a new employee possessing relevant skills can be recruited by the HR department. This will help the colleagues in HR department to see what are missing skills, whether skills are rather isolated or more transferable such as communication, problem-solving and self-control.

The document embeddings obtained from the GCN is shown in Figure 9. The first visualization where we investigate the document nodes itself looks quite random as the class labels are all mixed together. So, instead, we wanted to investigate them as an average of the skill vectors that they contain. That way, we obtain much better visualizations and there are clear clusters of labels in the visualization.

4.2.2 Dicecom dataset

Baseline model. The t-SNE visualization of the document embeddings of the baseline model applied to the Dicecom dataset is shown in Figure 10. Initial number of dimensions of the document vectors is 4,777. Note that it is a lot since the skills are not standardized in that dataset and there are a lot of unique words.

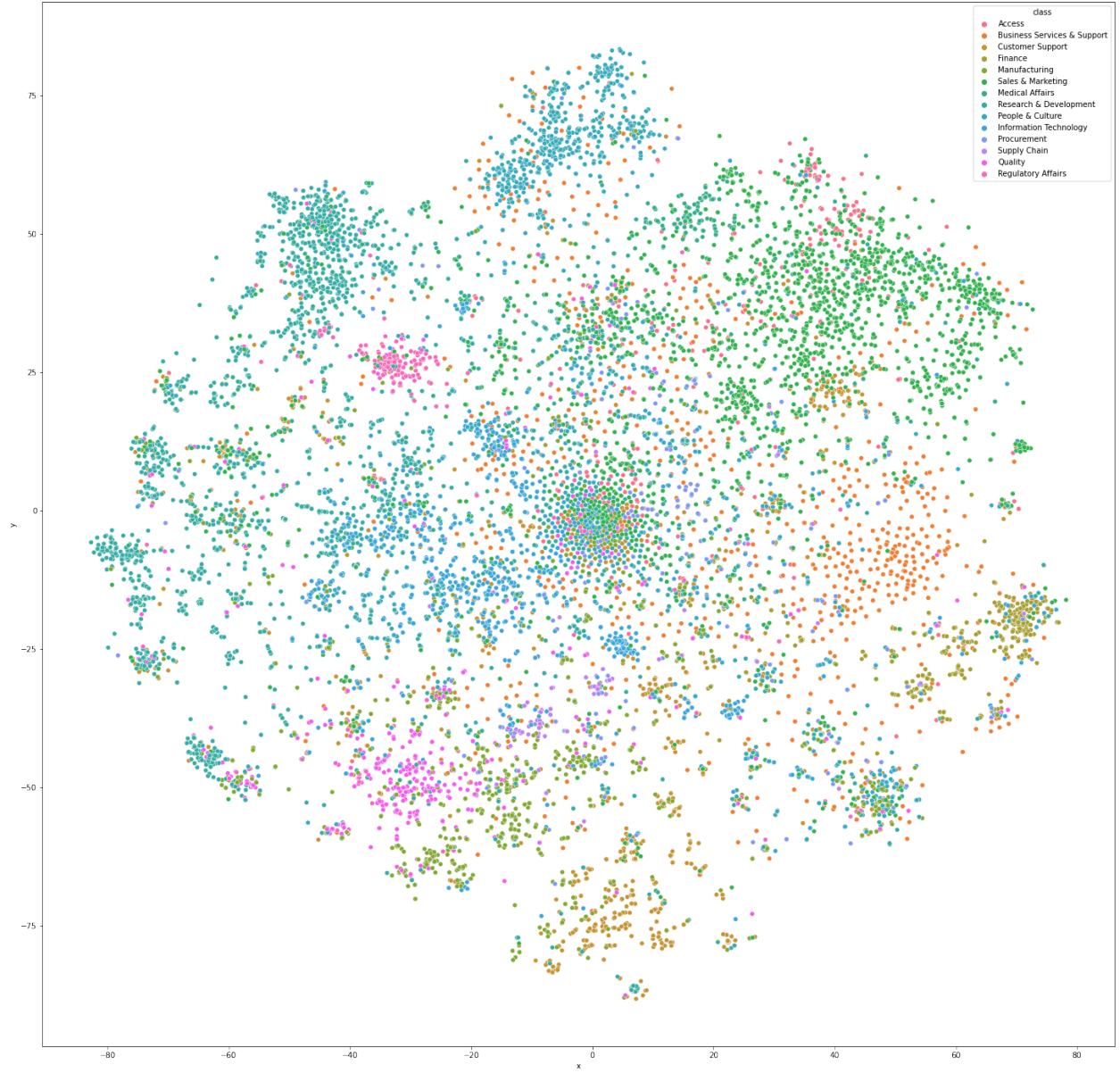


Figure 7: 2-dimentional t-SNE visualization of the document embeddings of the baseline model applied on the Roche dataset.

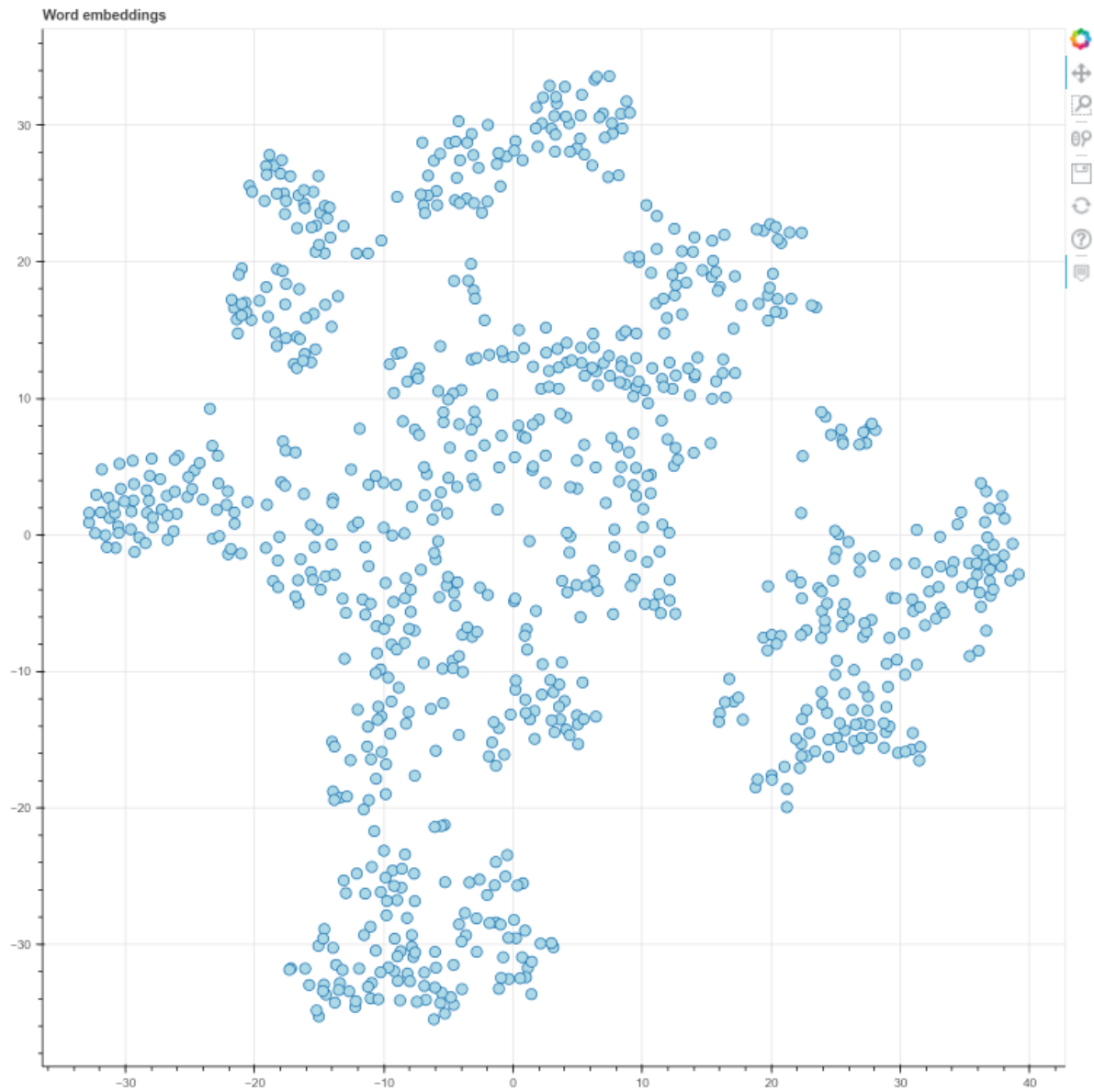
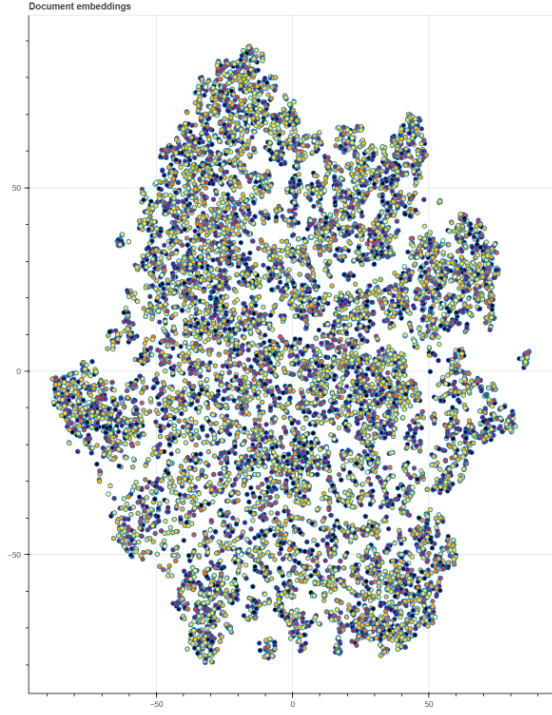
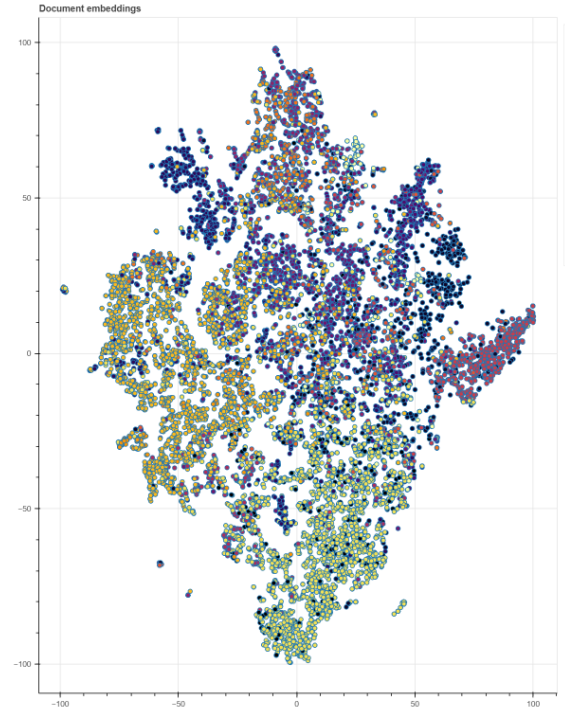


Figure 8: 2-dimentional t-SNE visualization of the skill embeddings of the GCN model applied on the Roche dataset.



(a) Document nodes



(b) Documents as average of skill nodes

Figure 9: 2-dimentional t-SNE visualizations of the document embeddings of the GCN model applied on the Roche dataset. Figure 9a shows the visualization of the embeddings at the document nodes in the graph directly whereas Figure 9b shows the visualization of the documents, where the t-SNE algorithm is applied on the average of the skill vectors possessed by the document.

Table 6: Classification report of the dummy classifier (stratified) for the Dicecom dataset.

	Precision	Recall	F1-score	Support
.net	0.13	0.13	0.13	303
business	0.14	0.15	0.15	240
java	0.21	0.21	0.21	381
network	0.15	0.16	0.15	192
project	0.13	0.15	0.14	252
software	0.21	0.19	0.20	419
accuracy			0.17	1787
macro avg	0.16	0.16	0.16	1787
weighted avg	0.17	0.17	0.17	1787

Table 7: Classification report of the baseline model for the Dicecom dataset.

	Precision	Recall	F1-score	Support
.net	0.64	0.53	0.58	303
business	0.79	0.83	0.81	240
java	0.82	0.81	0.82	381
network	0.61	0.72	0.66	192
project	0.85	0.76	0.80	252
software	0.64	0.88	0.66	419
accuracy			0.72	1787
macro avg	0.72	0.72	0.72	1787
weighted avg	0.72	0.72	0.72	1787

As we can see in the visualization, the documents are not really separated from each other well and there are no clear clusters.

Graph convolutional network. The initial number of dimensions of the document vectors is 6, which is the same as the number of distinct classes in the dataset.

The skill embeddings of the GCN model applied on the Dicecom dataset is shown in Figure 11. The skills are grouped nicely in this visualization. For example, in the lower left corner, there is a cluster of skills. When we investigate the content of it, we see skills such as "network", "cisco", "wan" (wide area network), "bgp" (border gateway protocol), "routers" and "firewall", which fit to each other nicely.

The document embeddings obtained from the GCN is shown in Figure 12. The first visualization where we investigate the document nodes itself looks quite random as the class labels are all mixed together. So, instead, we wanted to investigate them as an average of the skill vectors that they contain. That way, we obtain much better visualizations and there are clear clusters of labels in the visualization. We can observe that the labels in the upper right corner are kind of mixed with each other. It makes sense since those labels are "software" and "java", and a software engineering job can also require java as a skill. In that sense, it is not illogical that those classes are mixed with one another.

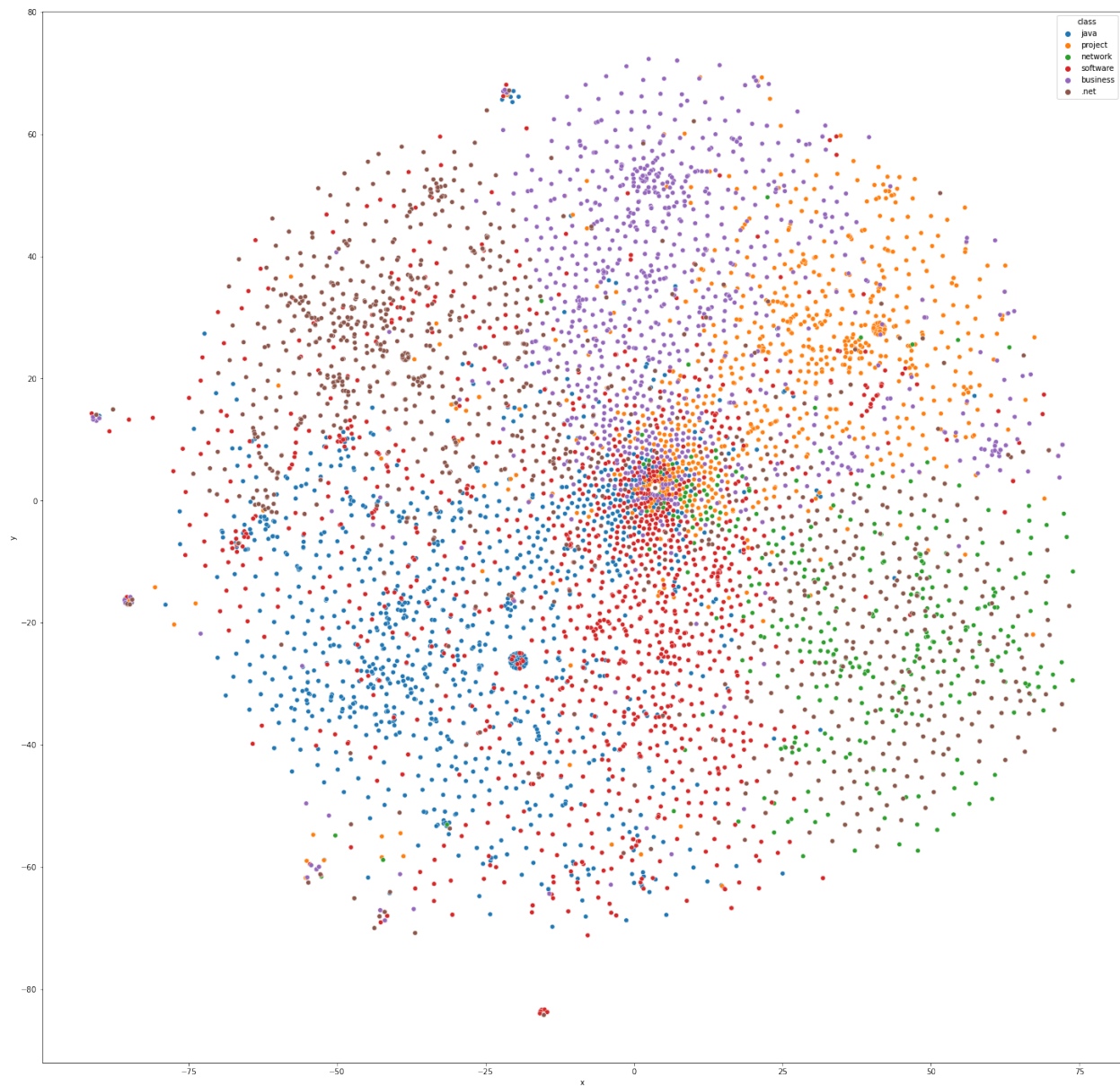


Figure 10: 2-dimentional t-SNE visualization of the document embeddings of the baseline model applied on the Dicecom dataset.

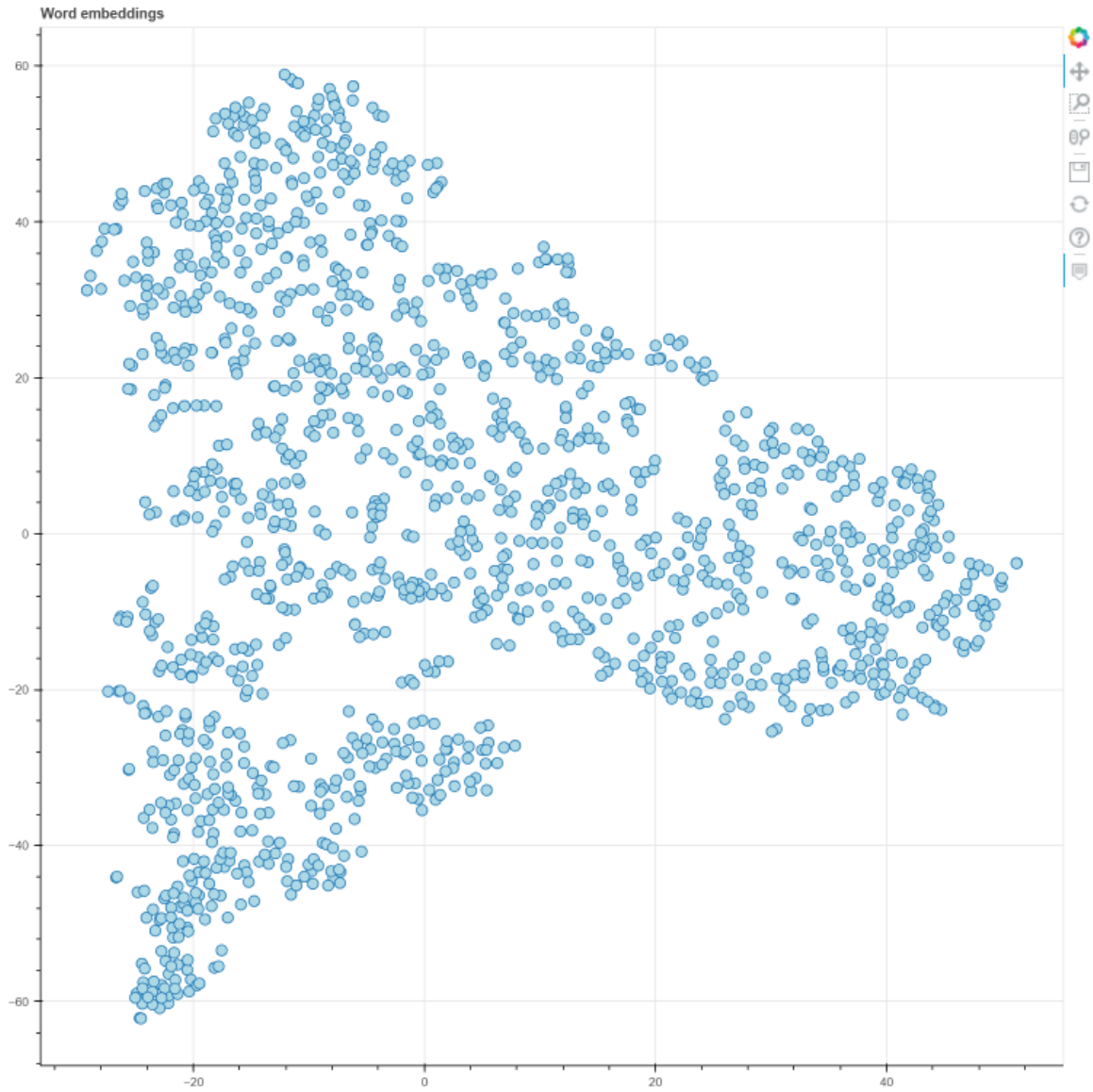
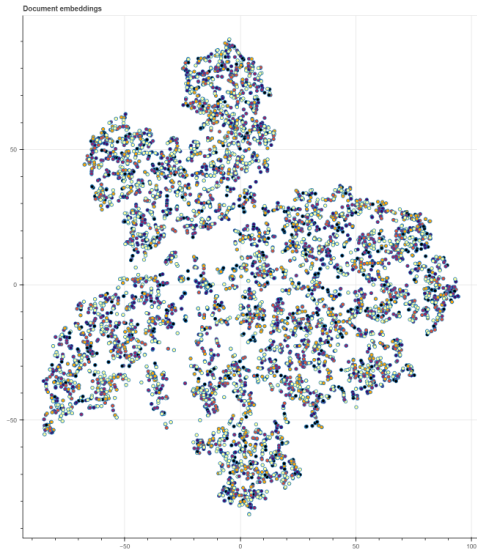
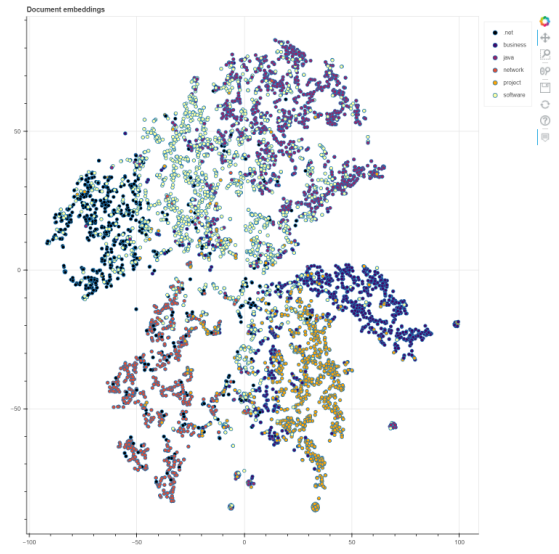


Figure 11: 2-dimensional t-SNE visualization of the skill embeddings of the GCN model applied on the Dicecom dataset.



(a) Document nodes



(b) Documents as average of skill nodes

Figure 12: 2-dimentional t-SNE visualizations of the document embeddings of the GCN model applied on the Dicecom dataset. Figure 12a shows the visualization of the embeddings at the document nodes in the graph directly whereas Figure 12b shows the visualization of the documents, where the t-SNE algorithm is applied on the average of the skill vectors possessed by the document.

Table 8: Classification report of the GCN model for the Dicecom dataset.

	Precision	Recall	F1-score	Support
.net	0.67	0.49	0.56	303
business	0.73	0.78	0.56	240
java	0.82	0.75	0.78	381
network	0.59	0.74	0.66	192
project	0.82	0.75	0.78	252
software	0.60	0.69	0.64	419
accuracy			0.70	1787
macro avg	0.70	0.70	0.70	1787
weighted avg	0.70	0.70	0.70	1787

5 Improvements and Modifications

In this section, we would like to discuss which changes we applied to the original network and whether they yielded better results in terms of performance. Different approaches are discussed below:

1. **Take the whole skill itself in the graph:** Instead of investigating the skills on a word level like in the original paper, we considered the word itself. For example, we treated the skill "data science" as a single entity instead of treating it as "data" and "science" separately.
2. **Take the whole document as a single window:** Instead of having a window of size, e.g. 20 and moving it across the whole corpus, we took the whole document as a single window. One can also consider taking `window_size = ∞` in that case so that the document itself will be investigated instead of a portion of it. This is a better way in the case of Roche dataset since the order of skills does not matter and the skills are simply separated by commas. Also, there can be at most one skill of a kind in the document and there is no need to treat skills close to one another differently than skills separated by a distance.
3. **Use conditional probability in the adjacency matrix instead of the PMI value:** Instead of carrying out the PMI calculation as in paper, conditional probability calculation is used on the available windows (in our case, documents). In the conditional probability, we assign the link between two words i and j according to the formula $P(i|j) = \frac{\#(i,j)}{\#(j)}$, where $\#(i,j)$ is the number of times skills i and j appear together and $\#(j)$ is the number of times the skill j appears. Note that conditional probability is not symmetric unlike the PMI value.
4. **PMI with negative edges:** In the original implementation, the PMI calculations with negative edges are simply set to zero in the adjacency matrix. One modification that we considered was to leave the negative edges as it is in the adjacency matrix.
5. **Simple 1 between documents and skills:** One other modification that we tried was to simply put 1 if a skill is present in the document, 0 otherwise.
6. **Symmetric conditional probability:** As a symmetric orientation of the conditional probability mentioned in item 3, we use the arithmetic average $\frac{P(i|j)+P(j|i)}{2}$ as the symmetric edge between words i and j .

7. **Add more depths to the network:** This does not make the model better. Actually, it makes the training longer and the accuracy value lower. One simple explanation for it would be that each document has only one layer of neighbors: "GCN can capture information only about immediate neighbors with one layer of convolution. When multiple GCN layers are stacked, information about larger neighborhoods are integrated." Since our model has only one single layer of neighbors, we do not need multiple layers in the network. "Each GCN layer acts as a form of Laplacian smoothing in essence, which makes the features of nodes in the same connected component similar. Thereby, adding too many convolutional layers will result in the output features over-smoothed and make them indistinguishable" [Hu et al., 2019].

The accuracy values obtained from those methods are shown in Table 9. Note that the accuracy values are lower than the results shown in section 4 due to the fact that these methods are applied on the non-processed Roche dataset. Hence, some job family groups are still present in the dataset and it makes the accuracy value worse.

Table 9: Accuracy values obtained from different novel methods.

Method	Accuracy
item 1 & item 2	0.6342
item 3	0.6253
item 4	0.6419 (+%1.67)
item 5	0.6216
item 6	0.6339
Original method	0.6252

6 Conclusion and Future Work

In this project, we investigated if company skills can be modelled in a consistent way by means of graph convolutional networks. According to the results that we obtained, similar skills are grouped together. This way, we can relate them together and it will help people in the HR department in a company to do appropriate recruitment decisions or posting of the job advertisement.

The current implementation of the GCN has several shortcomings. First, it does not support the usage of directed graphs. Although we investigated a non-symmetric adjacency matrix in section 5, it required a longer training time and the results are not very reliable. [Tong et al., 2020] suggest to use the concept of GCN for directed graphs, which is worth investigating in a future work. Second, the current structure of the graph is too shallow, which prevents us from getting global information about the skills in the graph. For this reason, it would be an interesting idea to investigate deeper graphs where some hierarchical information about the skills is present. [Hu et al., 2019] investigate this concept in their paper, and it would be an interesting direction to go in a future work. Third, the current implementation of GCN is a transductive learning method, meaning that the end result is not a predictive model but the entire train and test documents are used to build the graph. Hence, the model will not be able to predict the label of any additional data point to the network. For this reason, the entire procedure of building the graph and training the model has to be repeated in order to add new data to the existing graph.

References

- [Bergstra et al., 2015] Bergstra, J., Komer, B., Eliasmith, C., Yamins, D., and Cox, D. D. (2015). Hyperopt: a python library for model selection and hyperparameter optimization. *Computational Science & Discovery*, 8(1):014008.
- [Hu et al., 2019] Hu, F., Zhu, Y., Wu, S., Wang, L., and Tan, T. (2019). Hierarchical graph convolutional networks for semi-supervised node classification. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence, IJCAI’19*, page 4532–4539. AAAI Press.
- [Kingma and Ba, 2017] Kingma, D. P. and Ba, J. (2017). Adam: A method for stochastic optimization.
- [Kipf and Welling, 2017] Kipf, T. N. and Welling, M. (2017). Semi-supervised classification with graph convolutional networks.
- [Mikolov et al., 2013] Mikolov, T., Chen, K., Corrado, G., and Dean, J. (2013). Efficient estimation of word representations in vector space.
- [Pedregosa et al., 2011] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- [Tong et al., 2020] Tong, Z., Liang, Y., Sun, C., Rosenblum, D. S., and Lim, A. (2020). Directed graph convolutional network. *ArXiv*, abs/2004.13970.
- [van der Maaten and Hinton, 2008] van der Maaten, L. and Hinton, G. (2008). Visualizing high-dimensional data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605.
- [Van-Duyet et al., 2019] Van-Duyet, L., Quan, V. M., and An, D. Q. (2019). Skill2vec: Machine learning approach for determining the relevant skills from job description.
- [Yao et al., 2018] Yao, L., Mao, C., and Luo, Y. (2018). Graph convolutional networks for text classification.